

Sequence Processing with Recurrent Networks

Time will explain.
Jane Austin, *Persuasion*

In Chapter 7, we explored feedforward neural networks along with their applications to neural language models and text classification. In the case of language models, we saw that such networks can be trained to make predictions about the next word in a sequence given a limited context of preceding words — an approach that is reminiscent of the Markov approach to language modeling discussed in Chapter 3. These models operated by accepting a small fixed-sized window of tokens as input; longer sequences are processed by sliding this window over the input making incremental predictions, with the end result being a sequence of predictions spanning the input. Fig. 9.1, reproduced here from Chapter 7, illustrates this approach with a window of size 3. Here, we’re predicting which word will come next given the window *the ground there*. Subsequent words are predicted by sliding the window forward one word at a time.

Unfortunately, the sliding window approach is problematic for a number of reasons. First, it shares the primary weakness of Markov approaches in that it limits the context from which information can be extracted; anything outside the context window has no impact on the decision being made. This is problematic since there are many language tasks that require access to information that can be arbitrarily distant from the point at which processing is happening. Second, the use of windows makes it difficult for networks to learn systematic patterns arising from phenomena like constituency. For example, in Fig. 9.1 the phrase *the ground* appears twice in different windows: once, as shown, in the first and second positions in the window, and in the preceding step in the second and third slots, thus forcing the network to learn two separate patterns for a single constituent.

The subject of this chapter is recurrent neural networks, a class of networks designed to address these problems by processing sequences *explicitly as sequences*, allowing us to handle variable length inputs without the use of arbitrary fixed-sized windows.

9.1 Simple Recurrent Networks

A recurrent neural network is any network that contains a cycle within its network connections. That is, any network where the value of a unit is directly, or indirectly, dependent on its own output as an input. In general, such networks are difficult to reason about, and to train. However, within the general class of recurrent networks there are constrained architectures that have proven to be extremely useful when

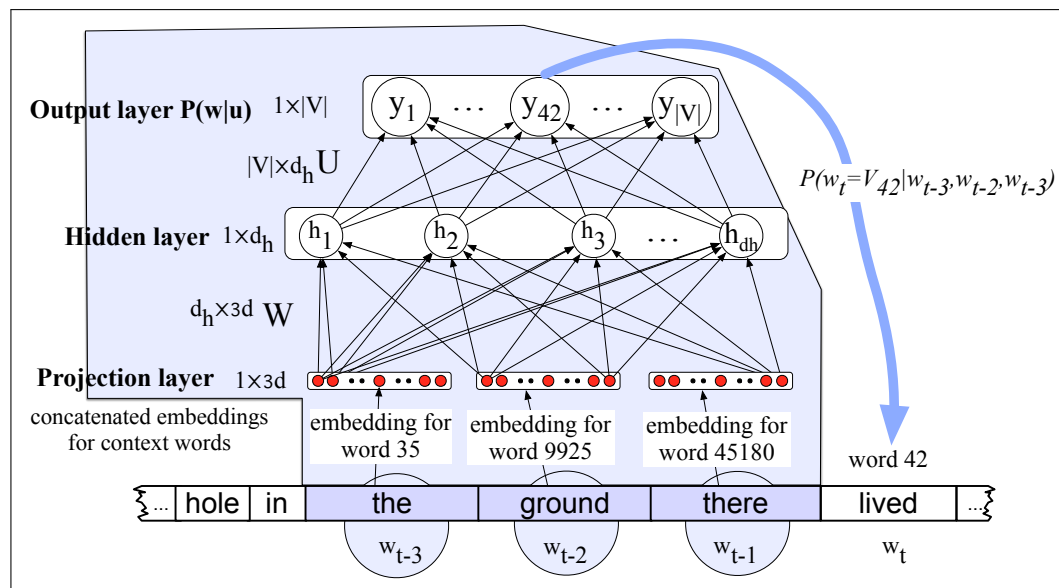


Figure 9.1 A simplified view of a feedforward neural language model moving through a text. At each timestep t the network takes the 3 context words, converts each to a d -dimensional embeddings, and concatenates the 3 embeddings together to get the $1 \times Nd$ unit input layer x for the network.

Simple
Recurrent
Networks

applied to language problems. In this section, we'll introduce a class of recurrent networks referred to as **Simple Recurrent Networks (SRNs)** or **Elman Networks** (Elman, 1990). These networks are useful in their own right, and will serve as the basis for more complex approaches to be discussed later in this chapter and again in Chapter 22.

Fig. 9.2 abstractly illustrates the recurrent structure of an SRN. As with ordinary feed-forward networks, an input vector representing the current input element, x_t , is multiplied by a weight matrix and then passed through an activation function to compute an activation value for a layer of hidden units. This hidden layer is, in turn, used to calculate a corresponding output, y_t . Sequences are processed by presenting one element at a time to the network. The key difference from a feed-

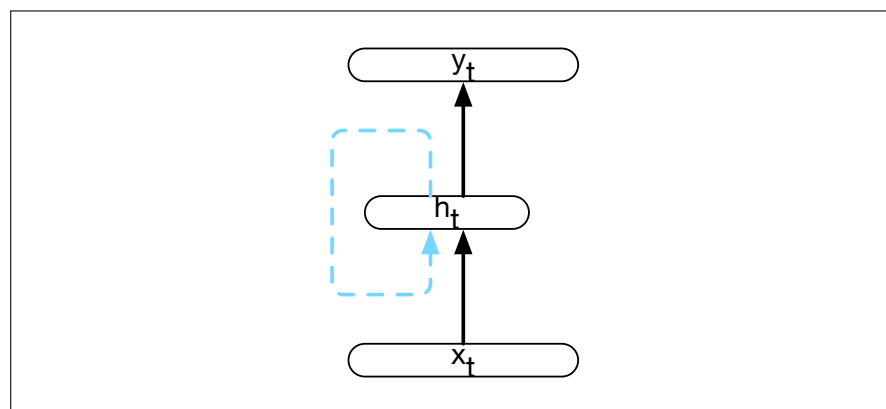


Figure 9.2 Simple recurrent neural network after Elman (Elman, 1990). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous timestep.

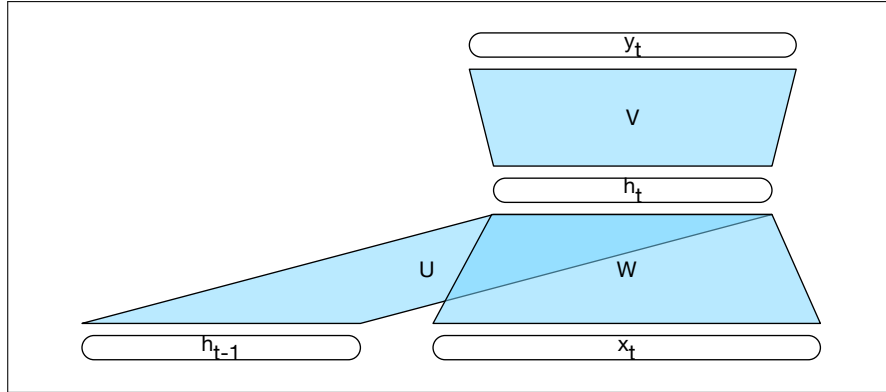


Figure 9.3 Simple recurrent neural network illustrated as a feed-forward network.

forward network lies in the recurrent link shown in the figure with the dashed line. This link augments the input to the hidden layer with the activation value of the hidden layer *from the preceding point in time*.

The hidden layer from the previous timestep provides a form of memory, or context, that encodes earlier processing and informs the decisions to be made at later points in time. Importantly, the architecture does not impose a fixed-length limit on this prior context; the context embodied in the previous hidden layer includes information extending back to the beginning of the sequence.

Adding this temporal dimension makes recurrent networks appear to be more exotic than non-recurrent architectures. But in reality, they're not all that different. Given an input vector and the values for the hidden layer from the previous time step, we're still performing the standard feed-forward calculation. To see this, consider Fig. 9.3 which clarifies the nature of the recurrence and how it factors into the computation at the hidden layer. The most significant addition lies in the new set of weights, U , that connect the hidden layer from the previous timestep to the current hidden layer. These weights determine how the network should make use of past context in calculating the output for the current input. As with the other weights in the network, these connections will be trained via backpropagation.

9.1.1 Inference in Simple RNNs

Forward inference (mapping a sequence of inputs to a sequence of outputs) in an SRN is nearly identical to what we've already seen with feedforward networks. To compute an output y_t for an input x_t , we need the activation value for the hidden layer h_t . To calculate this, we compute the dot product of the input x_t with the weight matrix W , and the dot product of the hidden layer from the previous time step h_{t-1} with the weight matrix U . We add these values together and pass them through a suitable activation function, g , to arrive at the activation value for the current hidden layer, h_t . Once we have the values for the hidden layer, we proceed with the usual computation to generate the output vector.

$$\begin{aligned} h_t &= g(Uh_{t-1} + Wx_t) \\ y_t &= f(Vh_t) \end{aligned}$$

In the commonly encountered case of soft classification, finding y_t consists of a softmax computation that provides a normalized probability distribution over the

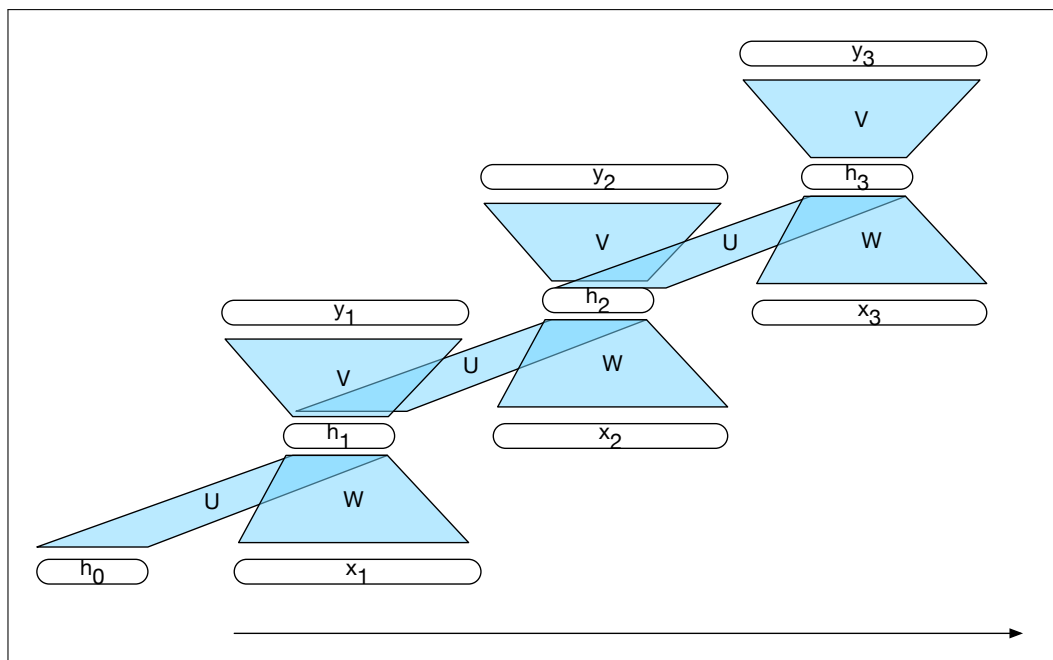


Figure 9.4 A simple recurrent neural network shown unrolled in time. Network layers are copied for each timestep, while the weights U , V and W are shared in common across all timesteps.

possible output classes.

$$y_t = \text{softmax}(Vh_t)$$

The sequential nature of simple recurrent networks can be illustrated by *un-rolling* the network in time as is shown in Fig. 9.4. In figures such as this, the various layers of units are copied for each time step to illustrate that they will have differing values over time. However the weights themselves are shared across the various timesteps. Finally, the fact that the computation at time t requires the value of the hidden layer from time $t - 1$ mandates an incremental inference algorithm that proceeds from the start of the sequence to the end as shown in Fig. 9.5.

```

function FORWARDRNN( $x$ ,  $network$ ) returns output sequence  $y$ 

 $h_0 \leftarrow 0$ 
for  $i \leftarrow 1$  to LENGTH( $x$ ) do
     $h_i \leftarrow g(U h_{i-1} + W x_i)$ 
     $y_i \leftarrow f(V h_i)$ 
return  $y$ 

```

Figure 9.5 Forward inference in a simple recurrent network.

9.1.2 Training

As we did with feed-forward networks, we'll use a training set, a loss function, and backpropagation to adjust the sets of weights in these recurrent networks. As shown in Fig. 9.3, we now have 3 sets of weights to update: W , the weights from the input

layer to the hidden layer, U , the weights from the previous hidden layer to the current hidden layer, and finally V , the weights from the hidden layer to the output layer.

Before going on, let's first review some of the notation that we introduced in Chapter 7. Assuming a network with an input layer x and a non-linear activation function g , we'll use $a^{[i]}$ to refer to the activation value from a layer i , which is the result of applying g to $z^{[i]}$, the weighted sum of the inputs to that layer. A simple two-layer feedforward network with W and V as the first and second sets of weights respectively, would be characterized as follows.

$$\begin{aligned} z^{[1]} &= Wx \\ a^{[1]} &= g(z^{[1]}) \\ z^{[2]} &= Ua^{[1]} \\ a^{[2]} &= g(z^{[2]}) \\ y &= a^{[2]} \end{aligned}$$

Fig. 9.4 illustrates the two considerations that we didn't have to worry about with backpropagation in feed-forward networks. First, to compute the loss function for the output at time t we need the hidden layer from time $t - 1$. Second, the hidden layer at time t influences both the output at time t and the hidden layer at time $t + 1$ (and hence the output and loss at $t + 1$). It follows from this that to assess the error accruing to h_t , we'll need to know its influence on both the current output *as well as the next one*.

Consider the situation where we are examining an input/output pair at time 2 as shown in Fig. 9.4. What do we need to compute the gradients needed to update the weights U , V , and W here? Let's start by reviewing how we compute the gradients required to update V (this computation is unchanged from feed-forward networks). To review from Chapter 7, we need to compute the derivative of the loss function L with respect to the weights V . However, since the loss is not expressed directly in terms of the weights, we apply the chain rule to get there indirectly.

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial V}$$

The first term is just the derivative of the loss function with respect to the network output, which is just the activation of the output layer, a . The second term is the derivative of the network output with respect to the intermediate network activation z , which is a function of the activation function g . The final term in our application of the chain rule is the derivative of the network activation with respect to the weights V , which is just the activation value of the current hidden layer h_t .

It's useful here to use the first two terms to define δ , an error term that represents how much of the scalar loss is attributable to each of the units in the output layer.

$$\delta_{out} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \quad (9.1)$$

$$\delta_{out} = L'g'(z) \quad (9.2)$$

Therefore, the final gradient we need to update the weight matrix V is just:

$$\frac{\partial L}{\partial V} = \delta_{out} h_t \quad (9.3)$$

Moving on, we need to compute the corresponding gradients for the weight matrices W and U : $\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial U}$. Here we encounter the first substantive change from

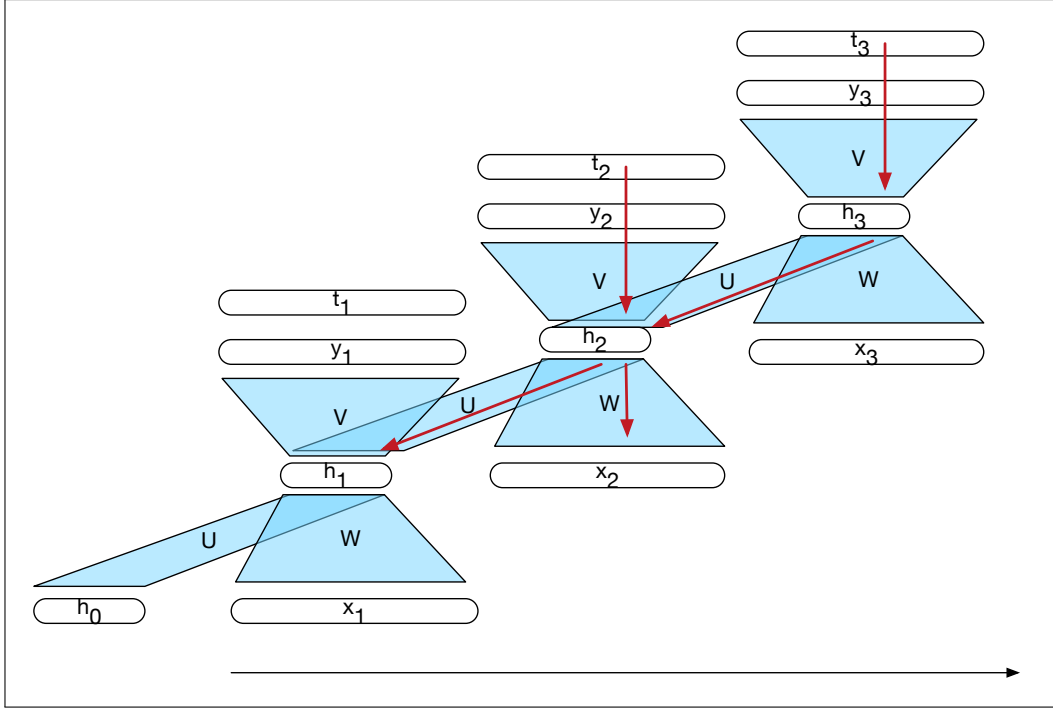


Figure 9.6 The backpropagation of errors in an SRN. The t_i vectors represent the targets for each element of the sequence from the training data. The red arrows illustrate the flow of backpropagated errors required to calculate the updates for U , V and W at time 2. The two incoming arrows converging on h_2 signal that these errors need to be summed.

feed-forward networks. The hidden state at time t contributes to the output and associated error at time t and to the output and error at the next timestep, $t + 1$. Therefore, the error term, δ_h , for the hidden layer must be the sum of the error term from the current output and its error from the next time step.

$$\delta_h = g'(z)V\delta_{out} + \delta_{next}$$

Given this total error term for the hidden layer, we can compute the gradients for the weights U and W in the usual way using the chain rule as we did in Chapter 7.

$$\begin{aligned}\frac{dL}{dW} &= \frac{dL}{dz} \frac{dz}{da} \frac{da}{dW} \\ \frac{dL}{dU} &= \frac{dL}{dz} \frac{dz}{da} \frac{da}{dU}\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W} &= \delta_h x_t \\ \frac{\partial L}{\partial U} &= \delta_h h_{t-1}\end{aligned}$$

These gradients provide us with the information needed to update the matrices U and W through ordinary backpropagation.

We're not quite done yet, we still need to assign proportional blame (compute the error term) back to the previous hidden layer h_{t-1} for use in further processing.

function BACKPROPTHROUGHTIME(*sequence, network*) **returns** gradients for weight updates
 forward pass to gather the loss
 backward pass compute error terms and assess blame

Figure 9.7 Backpropagation training through time. The forward pass computes the required loss values at each time step. The backward pass computes the gradients using the values from the forward pass.

This involves backpropagating the error from δ_h to h_{t-1} proportionally based on the weights in U .

$$\delta_{next} = g'(z)U\delta_h \quad (9.4)$$

At this point we have all the gradients needed to perform weight updates for each of our three sets of weights. Note that in this simple case there is no need to backpropagate the error through W to the input x , since the input training data is assumed to be fixed. If we wished to update our input word or character embeddings we would backpropagate the error through to them as well. We'll discuss this more in Section 9.5.

Taken together, all of these considerations lead to a two-pass algorithm for training the weights in SRNs. In the first pass, we perform forward inference, computing h_t , y_t , and an loss at each step in time, saving the value of the hidden layer at each step for use at the next time step. In the second phase, we process the sequence in reverse, computing the required error terms gradients as we go, computing and saving the error term for use in the hidden layer for each step backward.

Unfortunately, computing the gradients and updating weights for each item of a sequence individually would be extremely time-consuming. Instead, much as we did with mini-batch training in Chapter 7, we will accumulate gradients for the weights incrementally over the sequence, and then use those accumulated gradients in performing weight updates.

9.1.3 Unrolled Networks as Computational Graphs

We used the unrolled network shown in Fig. 9.4 as a way to understand the dynamic behavior of these networks over time. However, with modern computational frameworks and adequate computing resources, explicitly unrolling a recurrent network into a deep feed-forward computational graph is quite practical for word-by-word approaches to sentence-level processing. In such an approach, we provide a template that specifies the basic structure of the SRN, including all the necessary parameters for the input, output, and hidden layers, the weight matrices, as well as the activation and output functions to be used. Then, when provided with an input sequence such as a training sentence, we can compile a feed-forward graph specific to that input, and use that graph to perform forward inference or training via ordinary backpropagation.

For applications that involve much longer input sequences, such as speech recognition, character-by-character sentence processing, or streaming of continuous inputs, unrolling an entire input sequence may not be feasible. In these cases, we can unroll the input into manageable fixed-length segments and treat each segment as a distinct training item. This approach is called Truncated Backpropagation Through Time (TBTT).

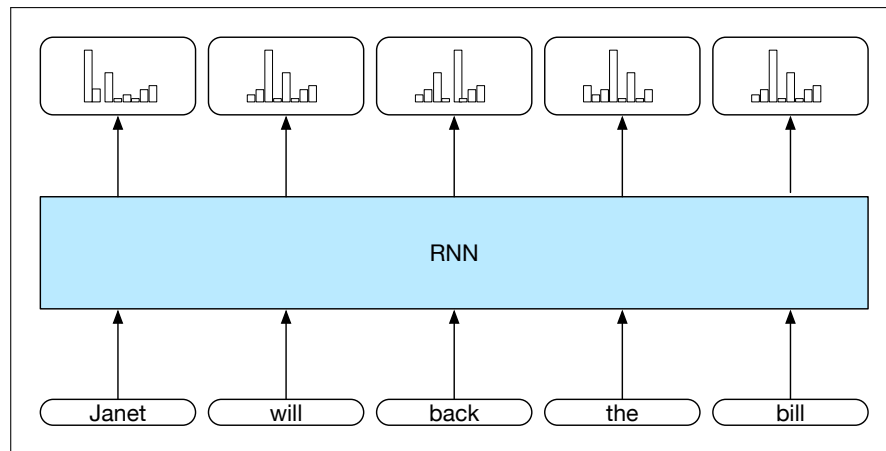


Figure 9.8 Part-of-speech tagging as sequence labeling with a simple RNN. Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

9.2 Applications of RNNs

Simple recurrent networks have proven to be an effective approach to language modeling, sequence labeling tasks such as part-of-speech tagging, as well as sequence classification tasks such as sentiment analysis and topic classification. And as we'll see in Chapter 22, they form the basic building blocks for sequence to sequence approaches to applications such as summarization and machine translation.

9.2.1 Generation with Neural Language Models

[Coming soon]

9.2.2 Sequence Labeling

In sequence labeling, the network's job is to assign a label to each element of a sequence chosen from a small fixed set of labels. The canonical example of such a task is part-of-speech tagging, discussed in Chapter 8. In a recurrent network-based approach to POS tagging, inputs are words and the outputs are tag probabilities generated by a softmax layer over the POS tagset, as illustrated in Fig. 9.8.

In this figure, the inputs at each time step are pre-trained word embeddings corresponding to the input tokens. The RNN block is an abstraction that represents an unrolled simple recurrent network consisting of an input layer, hidden layer, and output layer at each time step, as well as the shared U , V and W weight matrices that comprise the network. The outputs of the network at each time step represent the distribution over the POS tagset generated by a softmax layer. To generate an actual tag sequence as output, we can run forward inference over the input sequence and select the most likely tag from the softmax at each step. Since we're using a softmax layer to generate the probability distribution over the output tagset at each timestep, we'll rely on the cross entropy loss introduced in Chapter 7 to train the network.

A closely related, and extremely useful, application of sequence labeling is to find and classify *spans* of text corresponding to items of interest in some task domain. An example of such a task is **named entity recognition** — the problem of